Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos
Programação e Sistemas de Informação (3.º ano)
MÓDULO 10
PROGRAMAÇÃO ORIENTADA POR OBJETOS

Classes abstratas, métodos abstratos e classes seladas (sealed)

Parte I – Exemplo orientado

Uma loja online aceita vários tipos de pagamento. Todos os pagamentos têm um valor, mas cada tipo calcula a taxa de forma diferente:

- **MB WAY** – não tem taxa;
- **Cartão** – taxa de 1,5% sobre o valor.

Não faz sentido existir um «pagamento genérico»: qualquer pagamento é sempre de um tipo concreto. Por isso, a classe **Pagamento** é declarada como **abstrata** e serve apenas de superclasse.

```
// abstract: a classe Pagamento NÃO pode ser instanciada.
// Serve apenas de superclasse (modelo comum) para as subclasses.
abstract class Pagamento
{
    // Propriedade herdada por todas as subclasses
    public double Valor { get; set; }

    // Uma classe abstrata pode ter construtor: é chamado
    // pelas subclasses, através de : base(...)
    public Pagamento(double valor)
    {
        Valor = valor;
    }

    // Método abstrato: só tem a "assinatura", sem corpo { }.
    // Termina com ; e OBRIGA cada subclasse a implementá-lo.
    public abstract double CalcularTaxa();

    // Método normal: uma classe abstrata também pode ter código.
    // Este método é herdado tal como está pelas subclasses.
    public void MostrarResumo()
    {
        // CalcularTaxa() executa a versão da subclasse do objeto:
        // MB WAY devolve 0; cartão devolve 1,5% do valor.
        Console.WriteLine($"Valor: {Valor:F2} €");
        Console.WriteLine($"Taxa:  {CalcularTaxa():F2} €");
        Console.WriteLine($"Total: {Valor + CalcularTaxa():F2} €");
    }
}

// Subclasse: herda Valor e MostrarResumo() de Pagamento
class PagamentoMBWay : Pagamento
```

```
{
    // : base(valor) envia o valor para o construtor de Pagamento
    public PagamentoMBWay(double valor) : base(valor) { }

    // override: implementação obrigatória do método abstrato.
    // Sem este método, o programa não compila.
    public override double CalcularTaxa()
    {
        return 0;   // não tem taxa
    }
}

// sealed: classe fechada. Pode criar objetos desta classe,
// mas nenhuma outra classe pode herdar dela.
sealed class PagamentoCartao : Pagamento
{
    public PagamentoCartao(double valor) : base(valor) { }

    // Cada subclasse implementa o método à sua maneira
    public override double CalcularTaxa()
    {
        return Valor * 0.015;   // taxa de 1,5%
    }
}
```

Utilização no programa principal e resultado obtido:

```
// Criam-se objetos das subclasses (nunca de Pagamento)
PagamentoMBWay p1 = new PagamentoMBWay(40);
p1.MostrarResumo();   // método herdado da classe abstrata

PagamentoCartao p2 = new PagamentoCartao(40);
p2.MostrarResumo();
```

```
Valor: 40,00 €
Taxa:  0,00 €
Total: 40,00 €
Valor: 40,00 €
Taxa:  0,60 €
Total: 40,60 €
```

Aspetos a reter:

- `abstract class` – a classe **não pode ser instanciada**; só pode ser usada como superclasse.
- `abstract` num método – o método **não tem corpo** e **obriga** todas as subclasses a implementá-lo com `override`.
- **MostrarResumo()** é um método normal da classe abstrata que chama **CalcularTaxa()**: em execução, é usada a versão da subclasse a que o objeto pertence.
- `sealed class` – a classe pode ser instanciada, mas **não pode ter subclasses**.

- `: base(valor)` – envia o valor recebido para o construtor da superclasse.

Parte II – Experimentar e prever

1. Crie um projeto de consola, introduza o código do exemplo e confirme que obtém o resultado apresentado.
2. Acrescente a classe **PagamentoMultibanco**, com uma taxa fixa de 0,25 €. Antes de escrever o método **CalcularTaxa()**, compile o projeto. Que erro surge? Porquê?
3. Para cada excerto da tabela, preveja se o código compila. Quando não compilar, explique o motivo. Só depois confirme no Visual Studio.

	Excerto	**Compila? (S/N)**	**Justificação**
a)	`Pagamento p = new Pagamento(50);`		
b)	`Pagamento p = new PagamentoCartao(50);`		
c)	`class PagamentoEmpresa : PagamentoCartao { }`		
d)	`public abstract double CalcularTaxa() { return 0; }`		
e)	`class PagamentoPayPal : Pagamento { ... public double CalcularTaxa() { return 1; } }`		
f)	`abstract sealed class Teste { }`		

Nota: o excerto d) está escrito dentro da classe Pagamento; no excerto e), o construtor da classe foi omitido.

4. No excerto b), a variável é do tipo **Pagamento**, mas o objeto criado é um **PagamentoCartao**. Se executar `p.MostrarResumo();`, que taxa será apresentada? Justifique.

Parte III – Exercício: tipos de funcionários

Retomando a empresa da atividade anterior, os funcionários passam a ser de três tipos, e o salário bruto é calculado de forma diferente em cada um:

- **Efetivo** – recebe um vencimento base mensal;
- **Part-time** – recebe pelas horas trabalhadas (n.º de horas × valor por hora);

- **Estagiário** – recebe uma bolsa mensal fixa. A empresa decidiu que não podem existir subtipos de estagiário.

Utilize a seguinte estrutura de partida e complete-a, seguindo as indicações dos comentários:

```
// abstract: não será possível criar objetos Funcionario,
// apenas objetos dos tipos concretos (efetivo, part-time...)
abstract class Funcionario
{
    // Tarefa 5: criar as propriedades Numero e Nome

    // Tarefa 5: criar o construtor que recebe o número e o nome

    // Tarefa 6: declarar o método abstrato CalcularSalarioBruto()
    // Lembre-se: sem corpo { }, termina com ;
    // e obriga TODAS as subclasses a implementá-lo

    // Tarefa 7: criar o método normal ObterRecibo()
    // Devolve uma string e usa CalcularSalarioBruto():
    // cada objeto usará a versão da sua própria subclasse
}

// Subclasse: herda Numero, Nome e ObterRecibo()
class FuncionarioEfetivo : Funcionario
{
    // Acrescentar a propriedade VencimentoBase

    // Criar o construtor; usar : base(numero, nome) para enviar
    // estes dados para o construtor de Funcionario

    // Implementar CalcularSalarioBruto() com override,
    // que devolve o vencimento base
}

// Tarefa 8: criar a classe FuncionarioPartTime, seguindo o
// mesmo modelo (salário = horas trabalhadas x valor por hora)

// Tarefa 9: tornar esta classe fechada com sealed.
// Uma classe sealed pode ser instanciada, mas não pode ter
// subclasses: não existirão "subtipos" de estagiário.
class Estagiario : Funcionario
{
    // Acrescentar a propriedade Bolsa, o construtor e o override
}
```

5. Crie a classe abstrata **Funcionario**, com as propriedades **Numero** e **Nome**, e um **construtor** que as inicialize.
6. Na classe **Funcionario**, declare o método abstrato **CalcularSalarioBruto()**, que devolve um valor do tipo double.

7. Ainda na classe **Funcionario**, crie o método **ObterRecibo()** (não abstrato), que devolva um texto (string) com o número, o nome e o salário bruto do funcionário, utilizando o método **CalcularSalarioBruto()**.
8. Crie as subclasses **FuncionarioEfetivo**, **FuncionarioPartTime** e **Estagiario**, com as propriedades necessárias, e implemente em cada uma o método **CalcularSalarioBruto()** com override.
9. Declare a classe **Estagiario** como **sealed**. Para confirmar, tente criar uma classe **EstagiarioVerao** derivada de **Estagiario**, registe a mensagem de erro e, em seguida, remova-a.
10. No programa principal (consola), crie um objeto de cada tipo com os dados de teste seguintes e apresente os respetivos recibos. Tente também criar um objeto da classe **Funcionario** e registe o erro obtido.

Tipo	N.º	Nome	Dados	Salário bruto esperado
Efetivo	101	Ana Silva	Vencimento base: 1200,00 €	1200,00 €
Part-time	102	Rui Costa	80 horas × 8,50 €	680,00 €
Estagiário	103	Marta Lopes	Bolsa: 950,00 €	950,00 €

11. Utilize as classes criadas para implementar esta aplicação em Windows Forms:
    a. Inclua controlos radiobutton para escolher o tipo de funcionário, mostrando apenas os campos necessários para o tipo selecionado.
    b. Ao clicar num botão, crie o objeto do tipo escolhido e apresente o recibo devolvido pelo método **ObterRecibo()**.
    c. Valide os dados introduzidos (campos vazios, valores não numéricos ou negativos), apresentando mensagens adequadas.
12. **Desafio:** a empresa passou a ter comerciais, que recebem um vencimento base mais 5% do total de vendas do mês.
    a. Crie a classe **Comercial**. O que é que o compilador o obriga a fazer? Porquê?
    b. Teste com um vencimento base de 900,00 € e vendas de 6000,00 € (resultado esperado: 1200,00 €) e acrescente esta opção à aplicação.

Parte IV – Verificar a aprendizagem

Responda, por palavras suas, às seguintes questões:

- Porque é que faz sentido que a classe Funcionario seja abstrata e não uma classe normal?
- Uma classe abstrata pode ter métodos com código? Indique um exemplo desta ficha.
- Qual a diferença entre uma classe abstrata e uma classe sealed?
- Porque é que uma classe não pode ser, ao mesmo tempo, abstract e sealed?